

[journal]IEEEtran amsmath,amsfonts
algorithmic algorithm array [cap-
tion=false,font=normalsize,labelfont=sf,textfont=sf]subfig
textcomp stfloats url verbatim graphicx cite
multirow adjustbox [justification=centering]caption xr
appendices

An Evaluation and Comparison of GPU Hardware and Solver Libraries for Accelerating the OPM Flow Reservoir Simulator

Tong Dong Qiu, Andreas Thune, Vinícius Oliveira Martins, Markus Blatt, Alf Birger Rustad, Răzvan Nane

Abstract—Realistic reservoir simulation is known to be prohibitively expensive in terms of computation time when increasing the accuracy of the simulation or enlarging the model grid size. One method to address this issue is to parallelize the computation by dividing the model into several partitions and using multiple CPUs to compute the result using techniques such as MPI and multi-threading. Alternatively, GPUs are also a good candidate for accelerating computation due to their massively parallel architecture, which allows many floating point operations per second to be performed. Although Computational Flow Dynamics problems are complex and contain many computational parts that are challenging, the most difficult one is the execution of the numerical iterative solver that is used to solve the linear systems, which arise after discretizing the nonlinear system of equations that mathematically model the problem. The numerical iterative solver thus takes the most computational time and is challenging to solve efficiently due to the dependencies that exist in the model between cells. In this work, we evaluate the OPM Flow simulator and compare several state-of-the-art GPU solver libraries as well as custom-developed solutions for a BiCGStab solver using an ILU0 preconditioner and benchmark their performance against the default DUNE library implementation running on multiple CPU processors using MPI. The evaluated GPU software libraries include a manual linear solver in OpenCL and the integration of several third-party sparse linear algebra libraries, such as cuSparse, rocSparse, and amgcl. To perform our bench-marking, we use small, medium, and large use cases, starting with the public test case NORNE, which includes approximately 50k active cells, and ending with a large model that includes approximately 1 million active cells. We find that a GPU can accelerate a single dual-threaded MPI process up to 5.6 times and that it can compare with around 8 dual-threaded MPI processes.

Index Terms—I.6.8.h Parallel, B.2.1.b Parallel, B.2.2.a Simulation, C.1.2.g Parallel processors, C.1.2.j SIMD processors.

I. INTRODUCTION

COMPUTATIONAL Flow Dynamics (CFD) simulation is becoming extremely important to speed up prototyping and validating new designs for large-scale models, which are difficult to develop by a purely analytical method. For instance, reservoir simulators are used extensively by reservoir engineers nowadays to analyze the flow of fluids and predict oil and gas production. This applies both to managing current and developing new fields. The core of such a CFD simulation

is finding the solution to a set of partial differential equations (PDEs) constrained on some domain of interest. This is a two-step process, where the PDEs are first discretized over the defined domain using numerical techniques such as the finite-difference, -element, or -volume method, and second, solving the obtained linear and nonlinear systems using iterative linear solvers such as Krylov subspace solvers. However, solving the linear and nonlinear systems produced by these finite methods is usually time-consuming and complex because the linear systems are sparse and ill-conditioned. Furthermore, to improve the realism of the simulation, the number of cells required to be modeled is increasing at a steady pace. This leads to bigger systems, which in turn leads to higher run times. To accommodate these bigger systems and improve the scalability of the simulation, parallelization can be performed in two major directions: multi-core CPUs and many-core GPUs. In this work, we focus on the latter using the OPM project.

The Open Porous Media (OPM) [?] project encourages open and reproducible research for modeling and simulation of porous media. OPM supports several types of reservoir simulation, including black oil. The project is split into several modules, e.g., opm-simulators, and is built on top of state-of-the-art scientific libraries such as the DUNE [?] library to perform the computations in an efficient way. Therefore, it leverages powerful Krylov subspace preconditioners and solvers that are generally used to solve large-scale and sparse linear systems because such systems often take a lot of time to solve; for example, they may even take 90% or more of the simulation time for certain models. Efficient iterative solvers available in the DUNE library include GMRES and BiCGSTAB. To improve the convergence of an iterative solver, preconditioners are also used, with examples including AMG, domain decomposition, or multi-stage preconditioners [?], [?], and the well-known ILU0, which is the default one used in OPM Flow. The choice for ILU as the default is due to the better performance, even against industry reference simulators, achieved by avoiding non-contributing computations [?]. The Constrained Pressure Residual (CPR) [?] preconditioner is a special multi-stage preconditioner, only used for reservoir simulation. It solves the mostly elliptic pressure system using an AMG preconditioner. To support large-scale processing, the DUNE library supports partitioning and execution on multiple cores and CPU nodes using OpenMP and MPI. However, it does not support GPU acceleration, and considering the adoption of GPUs in recent years to speed up many compute-

T. D. Qiu is with Big Data Accelerate B.V., The Netherlands.

A. Thune is with Simula Research Laboratory, Norway.

V. O. Martins is with Institute of Mathematics, Statistics and Scientific Computation, Universidade Estadual de Campinas, Brazil.

M. Blatt is with OPM-OS AS, Norway.

A. B. Rustad is with Equinor S.A., Norway.

R. Nane is with Big Data Accelerate B.V., The Netherlands.

intensive tasks, the question that we ask ourselves is if OPM would be able to efficiently leverage such technology as well and execute faster if the iterative solvers would be run on such an accelerator.

GPUs have many small cores that can work together using the Same Instruction Multiple Data (SIMD) model. Each core performs the same instruction on different data. This allows the GPU to process massive amounts of data much faster than a CPU would, but only if the problem has enough parallelism. GPUs have shown large speedups in many different fields, like weather simulation [?], FEM-based structural analysis [?], numerically integrating ordinary differential equations (ODEs) [?], chip design [?], or machine learning with Tensorflow [?]. However, although GPUs have proven very useful for several other application domains, it is not clear if they are useful for the reservoir simulation domain considering the specific sparsity. Specifically, the different sparsity patterns and the dependencies between reservoir model cells cause code divergence, which essentially means that cells placed into independent partitions, due to dependencies, are still executed sequentially. To analyze how big this problem is, we develop, integrate, and perform a thorough evaluation with both manual OpenCL kernels and various third-party libraries targeted at different GPU hardware to accelerate the linear solver of OPM Flow [?] on a GPU. We target the default OPM ILU0 preconditioned BiCGStab linear solver used in OPM Flow. Concretely, the novelty of the paper is summarized as follows:

- Develop open-source manual OpenCL and CUDA kernels for ILU0 preconditioned BICGStab solver.
- Develop a custom open-source bridge to integrate the custom-developed solvers as well as third-party libraries into the OPM Flow simulator.
- Integrate both manual solvers and third-party libraries into OPM Flow, including amgcl, cuSparse, and roc-Sparse.
- Perform a thorough evaluation and comparison of all the developed and integrated libraries using three reservoir models with different sizes and running on GPU hardware from both Nvidia and AMD vendors. This is, to the best of our knowledge, the first work to present complete GPU results running on the open-source OPM Flow reservoir simulator using real-world use cases.

The paper is organized as follows. First, in section ??, we describe the background required to understand the OPM Flow simulator and the GPU preconditioners and iterative solver developed in this work. Then, in section ??, we highlight other interesting reservoir simulators and indicate what target processors they support. Section ?? describes the implementation details of how the different manual and third-party libraries were developed and integrated into OPM, including particularities specific to reservoir simulation, such as well contributions to the final solution. Section ?? presents the experimental results and discusses the obtained performance numbers for the different libraries and GPU hardware tested. Finally, section ?? summarizes the paper and lists future work.

II. BACKGROUND

IN this section, we will describe the OPM project and introduce the basic concepts used in the CFD field of reservoir simulation, such as grid, assembly, linear iteration, preconditioner, and linear solver. Also, concepts specific to reservoir simulation, such as the reservoir wells are defined.

A. OPM Project

Reservoir simulation [?], [?] uses mathematical models to predict the fluid flow dynamics in porous media. Typically, oil, water, and gas are modeled as fluids, and the porous media are usually rock or soil. The simulations can be used to improve reserve estimates, predict future production, or evaluate multiple reservoir management strategies. The first step is to create a static model of the reservoir, also called a geological model. This geological model is created by geologists and geophysicists by using multiple types of information, such as seismic data, well logs, production history, and rock properties. Important rock properties are type, porosity, water saturation, and permeability.

The Open Porous Media (OPM) [?] simulator, OPM Flow [?], is an open-source simulator that models black-oil [?] with dissolved gas and vaporized oil. Other characteristics of the model that can be modelled include rock-dependent capillary and relative-permeability curves, end-point scaling and hysteresis, and oil vaporization controls. The simulator's input file is compatible with the commercial simulator Eclipse [?], and the output file is readable by commercial post-processing software. The OPM project also features a post-processing software called ResInsight [?]. The black-oil model assumes three fluid phases (aqueous, oleic, and gaseous) and three components: water, oil, and gas. The oil and gas components represent all hydrocarbons in liquid and vapor form at standard conditions, respectively. Mixing is possible, so both oil and gas can be found in the oleic phase and gaseous phase. The partial differential equations (PDEs) are derived from the conservation of mass and Darcy's law [?], [?], together with suitable initial and boundary conditions. The equations give each grid element three unknowns. Additionally, the wells each have equations and unknowns. Choosing which unknowns to solve for is important. For non-miscible flow, the oil pressure p_o , water saturation s_w , and gas saturation s_g are chosen. For miscible flow, the gaseous phase may disappear if all the gas dissolves into the oleic phase. The oleic phase can also disappear if all the oil vaporizes into the gaseous phase. The oil pressure and water saturation are chosen, but the third variable is flexible:

$$x = \begin{cases} s_g, & \text{all three phases present,} \\ r_{go}, & \text{no gaseous phase,} \\ r_{og}, & \text{no oleic phase,} \end{cases} \quad (1)$$

with r_{go} being the ratio of dissolved gas to oil in the oleic phase, also called r_S in other literature, r_{og} being the ratio of vaporized oil to gas in the gaseous phase, also called r_V in other literature.

The PDEs need to be discretized to be solved numerically. They are discretized in space with an upwind finite-volume

scheme with a two-point flux approximation. Discretization in time is done using an implicit backward Euler scheme. The derived equations form a system of fully implicit nonlinear equations. This system is solved using a Newton-Raphson method and linearized. The resulting linear system is solved with a preconditioner linear solver. Figure ?? shows the general structure of OPM Flow.

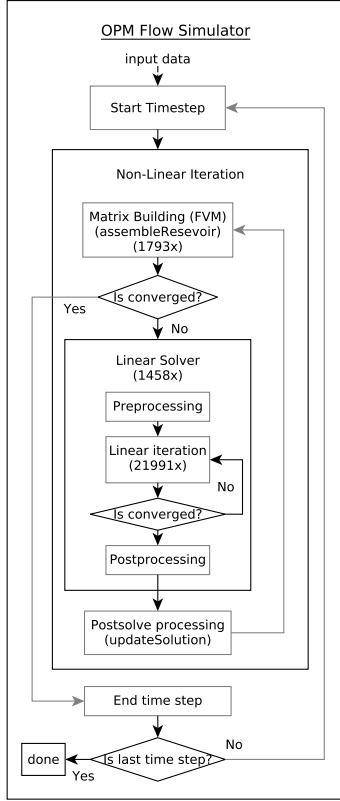


Fig. 1: The General Structure of OPM Flow.

The reservoir is modeled by a grid, where each cell/element has its own properties, such as porosity, volume, or transmissibility. The grid properties are defined in input files, which include structure, faults, and various static rock properties like porosity and permeability. OPM supports 1D, 2D, and 3D models, with three types of grids: Cartesian Regular, Radial, and Irregular Corner-Point. The first two types are relatively simple but are also rather limited. Irregular Corner-Point grids are the industry standard for describing the structure of complex reservoirs. The Cartesian Regular grid defines a regular orthogonal grid. For Irregular Corner-Point grids, coordinates lines or pillars are given to indicate x and y coordinates. Then, the top and bottom surfaces are specified by the z -coordinates of the cell's corner points along the four adjacent pillars. The cell then forms an irregular hexahedron, with each cell having the same outline as the cell above or below when viewed from above. Radial grids can be used to model radial flow near a wellbore. The Radial grids in OPM are currently modeled as 2D cylindrical grids because no flow in the θ direction is implemented. Grids cannot be combined in OPM, so Irregular Corner-Point is mostly used.

For more details on these grids, see the OPM Flow Reference Manual [?].

The simulation is fully implicit in time and has a flexible assembly of the linear system through automatic differentiation to enable rapid development of new fluid models. Traditionally, the closed-form expressions are obtained by differentiating the discretized flow equations by hand. This process is time-consuming and error-prone, but by using automatic differentiation, these drawbacks are removed. The simulator provides adaptive time step size controls. OPM Flow only uses Finite Volume Method (FVM), but the OPM project as a whole also implements Finite Element Method (FEM). The assembled linear system is blocked and sparse. The matrix is stored in the BCRSMMatrix (block compressed row storage) datastructure, provided by dune-istl [?]. Each block is a FieldMatrix, a dense matrix provided by dune-common. Due to fluid properties and other reservoir-specific parameters, the discretized equations could show elliptic, parabolic, or even hyperbolic behavior. The resulting linear system is usually non-symmetric and ill-conditioned. Instead of solving the system directly, the solution is approximated by a preconditioned iterative solver. The solver can either be bicgstab (default) or restarted gmres. For preconditioner, the options are ILU0 (default), AMG, or CPR.

OPM features two different well models: *standard* and *multi-segment*. A standard well has a single set of primary variables to describe the flow conditions inside. This works adequately for most wells and is, therefore, the default well model. For a three-phase black oil system, there are four primary variables: the weighted total flow rate, the weighted fractions of water and gas, and the bottom-hole pressure. Each well object has three variables in the implementation that are used during the linear solve: B , C , and D . B and C are $1 \times N_b$ blocked, sparse matrices, with $M \times N$ blocks, where N_b is the number of blockrows of the matrix, and M and N are 4 and 3 for blackoil respectively. D is a small $M \times M$ matrix. To apply a standard well, the solution vector x is a $C^T * (D^{-1} * (B * x))$. The multi-segment wells are used to simulate more advanced wells, such as multilateral wells, horizontal wells, and inflow control devices. The wellbore is divided into a number of segments, where each segment consists of a segment node and a flow path to the neighboring segment in the direction of the well head (outlet segment). Most segments also have inlet segment neighbors. Each segment has, in addition to the primary variables of a standard well, a node pressure variable.

Finally, table ?? lists the different OPM modules and their description.

B. ILU0

To solve the linear system $Ax = b$ efficiently, the matrix A is approximated by a factorization $A \approx LU$, with lower unitriangular matrix L and upper triangular matrix U . Once we obtained such a LU factorization, solving the linear system is performed according to the equations (3) and (4), which is equivalent to solving (1), both methods leading to finding the unknown x vector:

Name	Description
opm-common	reads Eclipse data files, provides Python bindings, build systems
opm-grid	provides interface for Dune-grid
opm-material	deprecated, now inside opm-common, provided infrastructure to handle material properties like relative-permeability/capillary pressure, thermodynamic relations, and empirical heat conduction laws
opm-models	contains fully-implicit numerical models for flow and transport in porous media
opm-simulators	contains simulator programs, like Flow, a fully implicit black-oil simulator that supports solvent and polymer options

TABLE I: Different OPM Modules and their Description.

$$Ax = b \quad (2)$$

$$LUx = b \quad (3)$$

$$Ly = b \quad (4)$$

$$Ux = y \quad (5)$$

The first equation (3) is solved using a forward substitution, and second, with the solution of y , a backward substitution is performed as in equation (4) to find x . The linear solves $Ly = b$ and $Ux = y$ are relatively easy since L and U are triangular matrices. In OPM terminology, this is called an ILU0 application, and the algorithm ?? highlights this process.

Algorithm 1 ILU0 application

Input: vector x , vector b , matrix L , matrix U

Output: updated vector x

// forward substitution

// backward substitution

For ILU0, L and U have the same sparsity pattern as matrix A . To find the L and U matrixes, a simple and sequential method to perform the decomposition is used, which is listed in Algorithm ??.

Algorithm 2 ILU0 decomposition

Input: matrix A

Output: matrix A , but contains both L and U

// nb is the number of blockrows of A

```

for r = 0 to nb:
  d = 1/arr
  for i = r+1 to nb:
    if (i,r) exists in S:
      e = d*air
      air = e
    for j = r+1 to nb:
      if (i,j) and (r,j) exist in S:
        aij = aij - e*arj

```

C. GPU Architecture and Programming Model

GPUs are traditionally designed for display purposes. Since each pixel is independent of others, they can be processed in parallel. This means the GPU has a parallel architecture. It can also process other algorithms that have parallelism.

GPUs have a massively parallel architecture, which allows them to process large amounts of data in a relatively short time if the processing algorithm has enough parallelism. The computation is done by stream processing. Data can be read, processed, and written at the same time in a pipeline. The two big manufacturers are NVIDIA and AMD. NVIDIA proprietary language CUDA is only available for NVIDIA GPUs, and the open-source OpenCL can run on both. For a kernel, a set of threads are launched, called workitems. Workitems are organized in workgroups. Workgroups can operate independently from each other, but the workitems in a workgroup are mapped to the same hardware area, a Streaming Multiprocessor (SM) on NVIDIA GPUs, or a Compute Unit (CU) on AMD GPUs. The workitems are also divided into wavefronts of 64 workitems. All 64 workitems execute the same instructions at the same time. If the kernel contains a branch that is not taken by all 64 workitems, the branch is serialized. Some of the workitems are deactivated, while the other take their branch; then activation is swapped, and the other branch is taken. For NVIDIA GPUs, the wavefronts consist of 32 workitems. Table ?? defines some of the OpenCL programming concepts and their CUDA counterparts.

It is important to only perform coalesced global memory accesses. The workitems all perform the same global memory read/write instruction, but with slightly different addresses. If the addresses are coalesced, the memory reads are combined into as many separate, serialized transactions as are needed. If all workitems read from the same memory chunk, it only needs one transaction.

OpenCL	CUDA
wavefront	warp
workgroup	(thread)block
workitem	thread
local memory	shared memory
private memory	local memory
global memory	global memory
Compute Unit (CU)	Streaming Multiprocessor (SM)

TABLE II: Some OpenCL definitions and their CUDA counterparts.

III. IMPLEMENTATION

IN this chapter, we present the design and implementation of the different preconditioners and solvers that we developed, and we provide details about how we integrated them and other third-party GPU libraries into OPM Flow.

A. OPM Flow

OPM Flow is composed of two main parts: the assembly and the linear solve. The linear solve is performed by the dune-

istl [?] iterative solver template library. OPM uses an ILU0 preconditioned BiCGStab solver as the default configuration. When the wells are added to the matrix, it performs the standard bicgstab algorithm. When the wells are separate, the linear operation (spmv) in the bicgstab is replaced by a combination of an spmv and an operation to apply the wells. This operation is defined in opm-simulators. Dune-istl is able to handle blocks of all sizes.

During initialization, the bridge verifies that the implementation specified on the command line is available and creates the chosen backend solver.

Right before dune-istl is called, our bridge tries to solve the linear system. At this point, the linear system is stored in the dune-istl BCRS matrix and block vector format. The block vector has a contiguous array internally, storing all the values. This is easily copied to the GPU. The matrix has three components: nonzero values, row pointers, and column indices. The nonzeros are blocked; Figure ?? shows how they are stored in memory. Due to the construction of the matrix in opm-simulators, the nonzeros are stored contiguously. These are also easily copied to the GPU, but a check must be made to ensure they are indeed contiguous. The row pointers and column indices are not readily available. To get them, the matrix is iterated through, and the sparsity pattern is written to two contiguous arrays.

The linear system is solved via a preconditioned bicgstab solver, capable of handling 3x3 blocks. If the backend solver is unsuccessful, the call to dune-istl is still made; see subsection ??.

B. Bicgstab solver

The bicgstab solver needs some basic functions like spmv, axpy, norm, and dot. The spmv implementation is described in ??.

For the norm and dot, the GPU kernel only returns partial sums, which are added to the CPU. Each work item calculates the result of one row, and the workgroup reduces this to one value in local memory. The function `work_group_reduce_add()` from Opencl 2.0 was not available, since Opencl 1.2 needs to be supported for NVIDIA GPUs.

The default stopping condition in Dune is a relative reduction in error of 0.01, with a maximum number of linear iterations of 200. Our implementation allows us to pass the same stopping criteria with the same default values.

C. Sparse Matrix-Vector multiplication

A sparse matrix-vector multiplication (spmv) multiplies a sparse matrix A with a dense vector x , resulting in a dense vector y . Each row i of A can be seen as a sparse vector, and then the inner product between row i and vector x can be calculated to form entry y_i . Every element of y can be calculated in parallel since they do not have dependencies. For OPM, the elements of A are actually small, dense blocks of size $N_x \times N$, and elements of x and y are small, dense vectors of size N . The sparse inner product becomes more complex: the two scalar elements a_{ij} and x_j are now a dense matrix and

a dense vector. To multiply, a standard dense matrix-vector multiplication is performed.

To implement this on the GPU, Algorithm 1 from [?] is used: Each warp or workgroup is assigned to one or more blockrows. The warp then iterates through them until all are processed. For a particular block row, the warp covers $32/bs^2$ blocks at a time, where bs is the block size. For a block size of 3, the warp covers 3 blocks with 27 threads or workitems, and 5 are left idle. The thread's position inside a block does not change, it only moves to the same position in another block. Each thread has its own running sum, row (r), and column (c) inside the block. It multiplies its element from the block with the corresponding element of x , adding it to the running sum. It then moves to the next block, which is actually $32/bs^2$ blocks over. Once the warp has iterated through all blocks in the row, threads that have the same r in the block reduce their running sums into one combined sum, which is written to the output vector. The paper assumes the values inside the block are stored column-major, but the blocks in OPM are generated by Dune [?], which are row-major. This does not change the way the algorithm works; during reading the element of A , the row and column of a thread are simply swapped.

D. ILU0

ILU0 has two phases: decomposition/creation and application. During the decomposition, Incomplete LU factorization without fill-in is performed. To apply the preconditioner, two triangular solves have to be performed with the resulting L and U factors. Since the sparsity pattern does not change throughout the simulation, the information derived from it can be reused. The sparsity pattern dictates the amount of parallelism that can be extracted from it. There are two implemented options to extract parallelism: level scheduling (LS) and graph coloring (GS). For debugging purposes, performing the ILU sequentially is also an option. Level scheduling respects indirect dependencies, whereas graph coloring is more aggressive and does not. If row C depends on row B , and row B depends on row A , then row C indirectly depends on row A . With LS, row C will be processed after row A is done. With GC, these two rows will be assigned the same color and processed simultaneously. All rows that are processed in parallel are in the same level or color.

After extracting the parallelism, a reordered copy of the matrix is made. In here, the rows of the matrix are reordered, such that all rows in the same color are in contiguous memory. The column indices are reordered accordingly. The b is also reordered. To perform the GPU kernels, the technique in [?] was used. This paper describes an spmv operation, but this can also be used for the ILU0 decomposition and application. During the execution of the decomposition and application, each color is processed by a new kernel launch.

E. Block-Jacobi-ILU0

ILU0 is a powerful preconditioner and an important part of the linear solver. A drawback is that it is fairly slow, a large amount of time is spent in the decomposition and application phases, especially on GPU. Other parts of the linear solver

can be parallelized relatively easily, but ILU0 is an inherently sequential algorithm because the rows depend on each other. This can be solved by using LS or GC, but the ILU part of the solver can still be the bottleneck.

To further increase parallelism, dependencies can be removed from the decomposition matrix. The original matrix should still be used for the rest of the solver. Removing blocks from the matrix for the preconditioner should be done in a smart way to reduce the quality of the preconditioner as little as possible while still improving parallelism. When using N MPI processes to run Flow, the grid can be partitioned into N partitions. Each partition will be assembled and solved in parallel, with some extra computation and synchronization for the boundary between the partitions. The partitions are created according to the transmissibilities between cells, these indicate how tightly the cells are coupled. More information can be found in [?].

The GPU implementation of the Block-Jacobi-ILU0 preconditioner uses the same partitioning algorithm, but the number of partitions is set by the user. Since every row in the matrix corresponds to an active cell in the grid, it is easy to remove blocks that represent a connection between two cells belonging to different partitions. These functions are inspired by the work in [?]. This new matrix is then used for the ILU preconditioner. The sparsity pattern is analyzed, and since it contains fewer blocks, it can extract more parallelism. The convergence of the ILU preconditioner is not affected too much if the number of partitions is low enough and if the partitioning is done in a smart way (using the transmissibilities).

The Jacobi matrix has the same number of rows and columns, but less nonzero blocks. To allow the matrix nonzeros to be copied by a single GPU copy, they must be stored in contiguous memory. The Dune::BCRSMMatrix allocates contiguous memory when the maximum number of nonzeros is passed to the constructor. The number of nonzeros in the full matrix serves as this upper bound. For every linear solve, blocks must be copied from the full matrix to the Jacobi matrix. This can be done by iterating the two matrices simultaneously and copying a single block when a match is found in the sparsity pattern. Alternatively, since the nonzeros of both matrices are stored in contiguous memory, the indices of the blocks that need to be copied can be stored in a big vector *indices*. Each nonzero block gets its own entry in that vector. Copying then occurs in a single loop:

Algorithm 3 Copy indexes

```
for i in indices:
    jacobi_matrix_nnz[i] = full_matrix_nnz[indices[i]]
```

F. Well Contributions

The default CPU implementation does not actually perform a standard bicgstab solver. After the spmv operation, a separate function is performed to incorporate the contributions of all active wells. A runtime parameter exists to put these contributions inside matrix A itself, allowing a standard bicgstab solver to be used. This will increase the complexity of the matrix

and might deteriorate converge performance. This function is described in Algorithm ???. When the wellcontributions are kept separate, they are added like in Algorithm ???. The spmv from the bicgstab solver is put in a comment.

Algorithm 4 Adding the wellcontributions to matrix A

```
for well in wells:
    A = A - well.CT * well.D-1 * well.B
```

Algorithm 5 Adding the wellcontributions separately

```
y = A * x
for well in wells:
    t1 = well.B * x
    t2 = well.D-1 * t1
    y = y - well.CT * t2
```

For standard wells:

- D is a single block
- B and C have 1 row, with a nonzero block at (0,j) if this well has a perforation at cell j

For multisegment wells:

- nseg is the number of segments of that well
- D is a (nseg x nseg) dense block matrix
- B and C are (nseg x ncells) sparse block matrices, they have a block at (i, j) if this well has a perforation at cell j connected to segment i. The columns of B/C have no more than one nonzero block.

To allow separate wellcontributions for standard wells on the GPU, manual kernels have been written to apply them. The variables B, C, and D can be interpreted as sparse vectors since they have only one row. There are three contiguous arrays, which store the B, C, and D of all the wells. For D, the inverse D^{-1} is stored instead, since multiplying with the inverse is easier. These three arrays are copied to the GPU. The CUDA and OpenCL kernels for standard wells apply to all wells in parallel. For every well, one workgroup is launched.

The multisegment wells are more complex; they have more, irregular-sized data, and are applied using UMFPack [?] on the CPU. Their application is not implemented on the GPU; instead, the x and y vectors are copied to the CPU, where they are applied with UMFPack. The resulting y vector is then copied back to the GPU.

G. Amgcl

Amgcl [?], [?] is a header-only C++ library for solving (blocked) sparse linear systems. It features different preconditioners and iterative solvers. Different backends like OpenMP, CUDA, or OpenCL can be used. All these different configurations can be chosen at runtime. It can also connect to VexCL [?], ViennaCL, Eigen [?] and Blaze [?].

The memory layout for blocked matrices in amgcl is different from OPM, as described in Figure ???. This means that for every linear solve, the matrix must be transformed to be used by amgcl.

To use amgcl, the wellcontributions must be included in the matrix.

0	1	2	9	10	11	0	1	2	3	4	5
3	4	5	12	13	14	6	7	8	9	10	11
6	7	8	15	16	17	12	13	14	15	16	17

Fig. 2: Memory layout of blocks in OPM (left) and amgcl (right).

H. Rocalution

Rocalution [?] is a sparse linear algebra library. It focuses on utilizing fine-grained parallelism and is built on top of AMD's ROCm [?] ecosystem. Numerous iterative solvers, preconditioners, and sparse matrix formats are supported. It is designed to be able to run on both NVIDIA and AMD GPUs.

The memory format for blocked matrices is row-major, like Dune's BCRSMatix. However, the nonzero values inside the blocks must be column-major. This means that for every linear solve, every matrix block must be transposed before sending it to rocalution.

It does have a BlockJacobi preconditioner, similar to the one described in Subsection ???. However, it requires the use of GlobalMatrix, a distributed matrix that does not have an easy way to handle blocked matrices. It also doesn't have a way to determine the blocks manually. Using the transmissibilities to partition helps to reduce the loss in convergence power.

To use rocalution, the wellcontributions must be included in the matrix. It might be possible to extend the Operator class, and both apply spmv and add the wellcontributions in its apply() function, but this might break the bicgstab solver and has not been tried yet.

I. cuSPARSE

cuSPARSE [?] is a library with basic linear algebra functions used for handling sparse matrices. It is created by NVIDIA and is implemented on top of the CUDA runtime. Since the bicgstab solver also uses some dense vector functions, cuBLAS [?] is used in combination with cuSPARSE to create cusparseSolver.

A CUDA kernel is written to apply the standardwell contributions for cusparseSolver, so that including them in the matrix is not required. Multisegment wells are still applied on CPU.

J. Rocsparse

Rocsparse [?] is the sparse linear algebra implementation of the ROCm framework. It has the same interface as cusparse. Using the building blocks provided in the library and the dense vector functions from rocblas, a simple ILU0-BiCGStab solver can be constructed. The advantage of using rocsparse over rocalution is that the Block-Jacobi-ILU0 preconditioner can be used. It should also be easier to allow for separate wellcontributions applications.

K. Fallback

If the chosen BdaSolver fails to converge within the specified limits (default 200 linear iterations), the simulation will

fall back to the CPU Dune implementation. The ILU0 decomposition is performed, and the solver is started.

The ILU0 decomposition from the chosen BdaSolver might be able to be reused instead since it has already been performed. Only the copying added time and could be faster than the CPU ILU0 decomposition. Since fallbacks are relatively rare, this is not implemented.

IV. EXPERIMENTAL RESULTS

IN this section, we present the experimental results. We highlight first the benchmarks used, describing their characteristics and introducing the chosen metric format that we will use to report the performance results. Second, we provide details of the different experimental setups that we used in this work, including the used CPU and GPU hardware. Finally, after the complete set of performance numbers is presented, we end the section with a discussion about the bottlenecks we encountered, explaining potential solutions or future steps to remove those bottlenecks.

A. Benchmarks

We use three different benchmarks to perform the evaluations of the different libraries implemented and integrated into OPM. The first use case is the NORNE [?] case, a real-world reservoir off the coast of Norway. NORNE contains 44431 active cells, and the resulting matrix has 320419 blocks when the well contributions are included or 313133 when they are kept separate. The dataset is open-source and available in the opm-data or opm-tests repository. The second use case is based on a refined version of NORNE, in which the grid size is increased so that it contains 355407 active cells. Finally, the last benchmark used is a proprietary closed-source model containing more than 1 million active cells. Table ?? summarizes the benchmarks, providing more details about their characteristics.

Notation:

A (B+C+D+E), F, G, H | I, J

A: Total time (seconds)

B: Assembly time (seconds)

C: Linear solve time (seconds)

D: Update time (seconds)

E: Pre/post step (seconds)

F: Overall Linearizations

G: Overall Newton Iterations

H: Overall Linear Iterations

I: Number of colors

J: Number of times BdaSolver failed and used OPM solver instead

If only 1 number after the '|' is reported, it is the number of failed solves.

Fig. 4: Notation of results

OPM Flow reports results at the end of each simulation by printing them in the console. The multi-line, verbose run-times output of flow is shortened for space reasons in this paper and is summarized below such that one run only takes one line

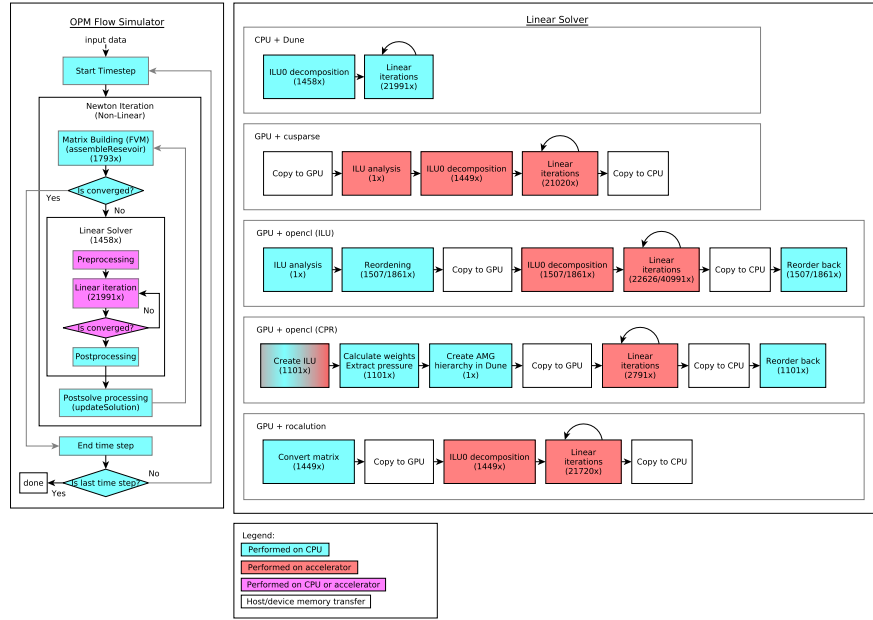


Fig. 3: The flowchart of different implementations.

Use Case	Active cells	Block Rows (N)	#NNZs	#NNZs/row	#Wells	Type Wells
NORNE	44431	44431	320419	7.21	3	STD
NORNE refined	355407	355407	2530093	7.11	3	STD
BigModel	1092723	1092723	6840701	6.26	5	MSW

TABLE III: Selected Benchmarks and their Characteristics.

Name	CPU	GPU
g001	Intel Xeon Platinum 8168 @ 2.7 GHz	NVIDIA V100-SXM3-32GB
g002	AMD EPYC 7763	NVIDIA A100-SXM-80GB
n013/n014	AMD EPYC 7763	NVIDIA A100-SXM-40GB
n004	AMD EPYC 7601	AMD Instinct MI100
n015/n016	AMD EPYC 7763	2x AMD Instinct MI210

TABLE IV: The Different Simula Nodes and their Hardware Configuration.

in a performance result table. The defined format is shown in Figure ?? . Furthermore, when a certain date is mentioned, the latest merge commit before that date/time is used to perform the run.

B. Experimental Setup

Simula [?] is a Norse research institute. Its main activities are research, innovation, and education. The research is conducted in five areas: communication systems, cryptography, machine learning, scientific computing, and software engineering. Simula hosts the Experimental Infrastructure for Exploration of Exascale Computing [?], [?] (eX³), a high-performance computing cluster. Some of its nodes are listed in Table ?? , while Table ?? highlights the specifications of the different hardware available in terms of the number of cores, maximum tera floating point operations per second (TFLOPS), maximum available memory on board, and the maximum bandwidth on the accelerator card.

C. Performance Results

First of all, to compare how the new hardware from Simula performs against our previous work, to compare the hardware

from [?] and the nodes from Simula, the release/2020.10-rc4 is benchmarked again in Table ?? . All four configurations are slower on the g001 node. This is surprising since the g001 node has a CPU with a faster base clock and a faster boost clock. Their GPUs have a different memory size, but the NORNE test case does not use enough memory to fill either. One possible explanation is that the Xeon E5-2698v4 has a cache of 50 MB, whereas the Platinum 8168 has 33 MB.

Please note that previously, we experimented only with a cusparse and OpenCL solvers using solely an NVIDIA GPU and using a linear solver that allowed only for coupled wellcontributions. In the following, we test multiple solvers using other libraries (i.e., rocalution, rocspase, and amgcl), we add an optimization technique for the ILU0 preconditioner, we extend the experimental results to include also AMD GPU hardware, and finally, we develop support for both coupled and separate for the well contributions as well as different types of wells in the GPU linear solvers. Nevertheless, because of the limited flexibility to modify a library linear solver, the separate well contributions are not fully supported for all the libraries, as explained in Section ?? Implementation. Table ?? summarizes the type of well (standard (STD) or multisegment

Name	Num. cores ¹	Max. FP64 TFLOPS	Memory size (GB)	Memory bandwidth (GB/s)
NVIDIA Tesla V100	5120	7.8	32	900
NVIDIA Tesla A100	6912	9.7	40/80	2000
AMD Instinct MI100	7680	11.5	32	1229
AMD Instinct MI210	6656	22.6	64	1638
AMD Instinct MI250	13312	45.3	128	3277

TABLE V: Different GPUs and their Specifications. ¹ NVIDIA CUDA Cores are not the same as AMD Stream Processors.

coupled	none	571 (152+319+63), 1793, 1458, 21196	
	cusparse	371 (151+123+62), 1783, 1449, 21010	0
	opencl LS	559 (156+303+64), 1844, 1507, 22626	0
	opencl GC	510 (180+218+76), 2109, 1774, 40863	0

TABLE VI: g001, Tesla V100, release/2020.10-rc4

Type Well / Library	STDWELL	MSWELL
Dune	yes	yes
cusparse	yes (GPU)	yes (CPU)
opencl	yes (GPU)	yes (CPU)
rocspase	yes (GPU)	yes (CPU)
rocalution	no	no
amgcl	no	no

TABLE VII: Summary of How the Types of Wells and Their Contributions to the System Matrix are Supported by the OPM Flow (GPU) Backends.

(MS)) and its support in the different GPU libraries.

Table ?? highlights the different solver libraries we evaluated and that we run on different CPU and GPU hardware. For space reasons, we highlight only the linear solver time and do not report the total time it took to run a complete reservoir simulation with OPM flow. However, the complete results of the runs are shown in Tables ??-??, ??-??, and ??-?? for the three use cases, respectively. Please note that due to space reasons, we limit the presentation only to the fastest results that were achieved, which for the GPU was the ROCm-based implementation, and we compare it against the multiple-rank scaled results using the DUNE library on the CPU.

where,

- *N/A* means the option is not available due to separate wells not being supported for that library,
- *Err1* is an OPM flow error related to the study case being too small to be distributed on 128 MPI processes,
- *Err2* OPM flow did not converge when using the default setting of chopping the time step ten times.
- *Slow* means the run was stopped before finishing due to taking a longer time compared to the software 1 MPI process, and
- *Same* means the run matches the mentioned library.

Analyzing the results, we find that a GPU-based ILU0-preconditioned BiCGStab linear solver can accelerate a single dual-threaded MPI process up to 5.6x times (3832s vs 685s) for a medium-size reservoir model and up to 3.3x (174s vs. 53s) for a small size model. For the medium-size model, i.e., NORNE refined, the performance is equivalent to approximately 8 MPI dual-threaded processes when using the same type of preconditioned solver. Therefore, we believe that when benchmarking a small-size model such as NORNE, which allows the model data to fit (almost) entirely into the CPU L1

caches, the benefits of using a GPU accelerator are minimal, as highlighted by the experimental results.

Comparing the AMD ROCm rocspase library solver with the NVIDIA CUDA corresponding implementation, we find that both perform equally well. Similarly, when we compare the older with the newer generation hardware device,s we notice an almost equal improvement in performance out-of-the-box, namely 22% for AMD in favor of the MI210 vs. MI100 and 15% for NVIDIA A100 vs. V100. When we analyze our fully manual implementation that uses manually written OpenCL kernel functions, which do not benefit from the highly optimized assembly-like kernel primitives, we notice a substantial slowdown of 4x (384s vs. 95s) against both the AMD ROCm rocspase library without Jacobi and the rocalution libraries. However, the benefit of relaxing the ILU0 preconditioner by enhancing it with the jacobi-based preprocessing and, hence, increasing the parallelism, is the highest in the case of the manual OpenCL implementation with a speedup of 3.8x (384s vs. 101s) when running on the more recent AMD hardware. Finally, we also evaluated the third-party open-source library amgcl and the experiments with an out-of-the-box configuration revealed a surprising 1.35x slowdown (235s vs 174s) when comparing to a single dual-threaded CPU run of flow, or in other words, a speedup of 2.5x (235s vs. 94s) in favor of the AMD ROCm rocalution based solver. Please note that the aforementioned evaluation is considering only the NORNE benchmark. However, an equivalent analysis can be performed for the other two use cases with approximately similar conclusions. This is left as an exercise for the reader because this can be easily inferred from Table ??.

Tables ??-?? to ??-?? show the complete performance results of a complete OPM flow reservoir simulation, including the times for the assembly, system update, and the pre- and post-processing time, as well as the number of iterations performed in the linear solver following the format described in Figure ?. Due to space reasons, we highlight in these tables only the runs with different MPI processes and the AMD ROCm rocspase solvers that performed the best among the GPU solutions tested. Please note that the GPU implementations do not currently support multiple ranks and, as such, cannot benefit from parallel multi-rank assembly, update, and post-processing that further improves the performance of multiple CPU MPI ranks experiments vs the GPU ones because the assembly is performed on a single CPU when a GPU solver is chosen.

Library Configuration	Benchmark:	NORNE		NORNE Refined		Big Model	
	Hardware Device(s)	Coupled	Separate	Coupled	Separate	Coupled	Separate
DUNE, 1 MPI	EPYC 7763	174	178	3832	3847	<i>Err2</i>	3758
DUNE, 2 MPI	EPYC 7763	86	82	2161	2142	<i>Err2</i>	1853
DUNE, 4 MPI	EPYC 7763	45	39	1283	1162	<i>Err2</i>	1097
DUNE, 8 MPI	EPYC 7763	31	26	696	618	<i>Err2</i>	593
DUNE, 16 MPI	EPYC 7763	19	17	424	329	<i>Err2</i>	359
DUNE, 32 MPI	EPYC 7763	19	17	291	255	<i>Err2</i>	251
DUNE, 64 MPI	EPYC 7763	16	14	267	212	<i>Err2</i>	215
DUNE, 128 MPI	EPYC 7763	<i>Err1</i>	<i>Err1</i>	341	277	<i>Err2</i>	281
rocsparse-0	MI210	95	91	779	873	908	804
rocsparse-150	MI210	53	58	685	867	1030	995
rocalution	MI210	94	N/A	1014	N/A	<i>same rocs</i>	N/A
opencl-0 LS	MI210	384	506	2096	2685	<i>slow</i>	<i>slow</i>
opencl-150 LS	MI210	101	147	1440	1289	<i>slow</i>	<i>slow</i>
amgcl, vexcl	MI210	235	N/A	4729	N/A	<i>slow</i>	N/A
amgcl, vexcl	V100	276	N/A	7645	N/A	<i>slow</i>	N/A
amgcl, vexcl	A100	195	N/A	4419	N/A	<i>slow</i>	N/A
amgcl, cuda	V100	685	N/A	Err	N/A	<i>slow</i>	N/A
amgcl, cuda	A100	772	N/A	Err	N/A	<i>slow</i>	N/A
cusparse-0	V100	95	86	1205	913	<i>same rocs</i>	<i>same rocs</i>
cusparse-0	A100	118	101	1025	976	<i>same rocs</i>	<i>same rocs</i>
cusparse-150	V100	60	59	1103	1238	<i>same rocs</i>	<i>same rocs</i>
cusparse-150	A100	57	51	807	845	<i>same rocs</i>	<i>same rocs</i>
rocsparse-0	MI100	101	108	950	1094	<i>same rocs</i>	<i>same rocs</i>
rocsparse-150	MI100	55	71	964	1170	<i>same rocs</i>	<i>same rocs</i>
rocalution	MI100	110	N/A	1211	N/A	<i>slow</i>	N/A
opencl-0 LS	MI100	429	254	3850	3465	<i>slow</i>	<i>slow</i>
opencl-150 LS	MI100	143	135	1753	1867	<i>slow</i>	<i>slow</i>
amgcl, vexcl 7	MI100	241	N/A	6006	N/A	<i>slow</i>	N/A

TABLE VIII: OPM Flow Linear Solver Time in Seconds Running on Different Hardware Devices Using the Masters from 2023-4-13 Configured with the Default ILU0 Preconditioned BiCGStab Solver.

D. Profile numbers

To understand the bottlenecks of the ILU0-preconditioned linear solver on the GPU, we perform several profiles using the profilers available in the different GPU vendors' toolboxes. That is, we use the NVIDIA Nsight Compute (ncu) profiler to gain insight into the OPM flow simulator for the fastest CUDA implementation, namely the cusparse ILU0 solver with block jacobi, and run it with settings for 0 and 150 jacobi blocks. We analyze the profile numbers for all the benchmarks for both cases when the well contributions are included in the matrix or treated separately. For the AMD GPU implementation, we select the rocsparse implementation with block jacobi running with the same configurations for three use cases using the rocprof profilers. Additionally, we profile the big model benchmark using the more high-level omniperf tool from the ROCM framework. The results of these profiles are shown in Tables ??-??, ??-??, and ?? for the three benchmarks. Please note that we do not report numbers for the complete preconditioned solver, but we select only the most important kernels in the preconditioner (i.e., the lower and upper triangular solves) and blocked scale (i.e., bsrsv2), a fully parallel operation, and blocked sparse matrix-vector multiplication (i.e., bsrsv) in the solver part to show the (in)efficiencies of the GPU implementation when scaling the model size.

The first thing we notice is the contrast between the scale_bsrsv2 operation and how this is able to utilize the resources efficiently when increasing the size of the model due to its inherent parallelism as opposed to the lower and upper solver that is the core of the ILU0 application, which

are not able to leverage the massive resources available on the GPU. For example, in terms of memory throughput, only 104 GB/s is achieved, which accounts for less than 10% of the available memory bandwidth being utilized. For the spmv kernel, this reaches 343 GB/s, which is more than 3x better, but it is still far from the maximum available. This is caused in part due to the sequential nature of the ILU0 application and the irregular memory accesses in the spmv. Furthermore, we see that when increasing the parallelism in the ILU0 preconditioner using the Jacobi technique, the bandwidth is almost doubled from 118 GB/s to 196 GB/s, as shown in Table ???. However, this is still just a small percentage of the total maximum available bandwidth. Please note that due to issues with running omniperf for NORNE and NORNE modified, we report omniperf profile numbers only for the bigmod use case. In terms of resources, we notice that we have plenty of resources left on the board that are not utilized (especially for the lower and upper solves), which can be attributed to the fact that we do not have enough data to feed these resources since the sequential accesses and dependencies in the ILU0 nature prevent further parallelism.

Finally, to clearly match the above fine-metrics profile, we also perform course-grained profiles in the form of a run-time analysis by measuring the total time it takes to run the different parts in the GPU solver backend of the OPM flow simulator. We timed the different parts of the rocsparse GPU solver implementation, and we printed the accumulated timers at the end of the simulation. Table ?? highlights the output of this analysis. These numbers are grouped by the computationally different parts of the solver, such as the total time it takes to

Runtime metric	Benchmark:	NORNE		NORNE Refined		Big Model			
	Config Wells	coupled		coupled		coupled		separate	
	#Jac Blocks	0	150	0	150	0	150	0	150
copy_to_jacobiMatrix	-	3.4	-	55.3	-	126.9	-	113.7	-
check_zeros_on_diagonal	1.7	2.1	15.5	30.9	36.4	72.6	43.1	60.7	60.7
copy_to_GPU	5.7	6.7	48.1	84.1	128.7	208.5	112.2	205.5	205.5
decomp	4.7	1.4	49.9	19.4	90.7	68.6	36.7	26.0	26.0
total_solve	77.6	32.1	603.3	450.1	299.5	245.6	555.2	576.5	576.5
prec_apply	66.9	22.3	532	376	261.9	209.0	298.9	232.7	232.7
spmv	2.4	2.7	41.3	43	22.2	22.4	28.7	28.1	28.1
wells	-	-	-	-	-	-	209.1	297.1	297.1
rest	5.8	5	20.2	21	9.3	8.3	11.4	11.4	11.4
TOTAL time timers	89.7	45.7	716.8	639.8	555.9	722.8	740.1	975.2	975.2
Linear solve time (OPM)	91.2	47.1	780	671	1055.9	1304.5	781.8	1021.9	1021.9
linearize wells	0.6	0.6	3.9	4.1	476.7	460.3	1.8	1.7	1.7

TABLE IX: OPM Flow Linear Solver Time in Seconds Running on Different Hardware Devices Using the Masters from 2023-4-13.

Profiled Kernel	Benchmark:	Big Model	
	Profiled Metric	0 blocks	150 blocks
ilu_apply	Bandwidth(GB/s)	118	196
	LDS util (%)	30	37
	Vector L1 Data Cache Hit(%)	72	71
	Vector L1 Data Cache BW(%)	16	16
	Vector L1 Data Cache util(%)	99	98
	Vector L1 Coalescing(%)	56	51
	L2 Cache Hit (%)	84	69
	L2 Cache Util (%)	99	98
ilu_decomp	Bandwidth(GB/s)	30	57
spmv	Bandwidth(GB/s)	806	807

TABLE X: OPM Flow Profiles of the GPU rocSparse Linear Solver using the 0 and 150 Jacobi Blocks Preconditioner Settings on the AMD MI210 using the Omniperf Tool.

do the decomposition, how much it takes to perform spmv, as well as how long solving for the well contributions takes. Additionally, the copying and transfer from/to the GPU are included in this analysis. We observe that for the latter, these take a non-negligible amount and that they scale poorly with increasing size, which partly explains the lower relative speed-up in performance when comparing NORNE against NORNE refined, i.e., 89.7s vs 45.7s, which is almost 2x speedup, versus 716.8s vs 639.8s that is only a minor 12% speed-up. The trend is extended for the bigger model, in which case we even notice a slowdown for the more parallel 150-blocks ILU0 implementation. Therefore, one of the key conclusions to optimize is to reduce the cost for the memory transfers and copy to jacobi matrix. Furthermore, the different type of wells seems to contribute as well to the relative slowdown, and it will require attention.

V. CONCLUSION AND FUTURE WORK

IN this paper, we have evaluated the potential of using GPUs in the OPM project, which contains open-source implementations of several reservoir simulation CFD applications. To perform this study, we have developed several manual ILU0 preconditioned BiCGStab iterative solvers using OpenCL and CUDA and experimented with both AMD and NVIDIA GPUs. Furthermore, we have developed a bridge to seamlessly connect the GPU solvers and third-party scientific libraries into OPM. Finally, we provided extensive evaluation

results for all different solver configurations used running on different GPU hardware offered by different hardware vendors. To perform our bench-marking, we use small, medium, and large use cases, starting with the public test case NORNE, which includes approximately 50k active cells, and ending with a large model that includes approximately 1 million active cells. We find that a GPU can accelerate a single dual-thread MPI process up to 5.6 times and that it can compare with around 8 dual-thread MPI processes. Although the results are not impressive with respect to GPU performance in other fields that deal with structured computations with regular memory accesses, where massive speedups are observed, the unstructured and irregular nature of the computations in three-phase black oil reservoir simulations makes the GPU performance less impressive. Nevertheless, in this paper, we show that with well-suited techniques, we can improve the performance as it was shown when we compared it to the out-of-the-box library rocspare [?]. Finally, the source codes for the GPU kernels, solvers, and the integration with OPM *Flow* are made available in the git repositories of the OPM project [?].

In the future, we plan to further analyze the potential slowdowns in the GPU solvers that prevented us from obtaining more impressive speedup numbers. First and foremost, we need to continue with the profiling of the kernels and understand if the matrix data can be better encoded to maximize the cache hit and increase the bandwidth utilized. Furthermore, we should also investigate the impact of adding the well contribution to the matrix or the impact of the memory transfers when the contributions are calculated on the CPU. This could clarify why the speedup for the largest model is smaller than for the small-scale NORNE test case. Finally, several optimizations could be performed, such as using pinned memory in OPM for the assembly part to avoid the implicit CPU-to-CPU memory transfers of the system matrix before the data is copied to the GPU, and overlapping the copy of the original matrix to the GPU while the jacobi matrix is created on the CPU.

ACKNOWLEDGMENTS

We gratefully acknowledge the support of EPIC – Energy Production Innovation Center, hosted by the University of Campinas (UNICAMP) and sponsored by FAPESP – São Paulo Research Foundation (2017/15736-3). We acknowledge

the support of ANP (Brazil's National Oil, Natural Gas and Biofuels Agency) through the R&D levy regulation - Project reference number 24174-5. Special thanks are given to the "Centro de Estudos de Energia e Petróleo" (CEPETRO), School of Mechanical Engineering at UNICAMP and Institute of Mathematics, Statistics and Scientific Computation for their support throughout this research.